Building Data Apps In Python With Streamlit

Session 1: Setup & Basics

Ari Lamstein
AriLamstein.com

Exercise: What is a Data App?

- Type your answer to these 2 questions in the chat:
  - *What do you already know about Data Apps, Python and Streamlit?*
  - *What do you want to build after completing this course?*

About me

Ari Lamstein

- Started out as a PhD student in Computer Science -> left to join industry
- Worked as a software engineer at large tech companies in San Francisco
- Authored several R packages that deal with analyzing public datasets. R Trainer.
- Recent years: Python.
- Created two data apps using Streamlit:
  - Covid Demographics Explorer
  - Immigration Enforcement Explorer

Before I give my own answer about what a "data app" is, let me introduce myself

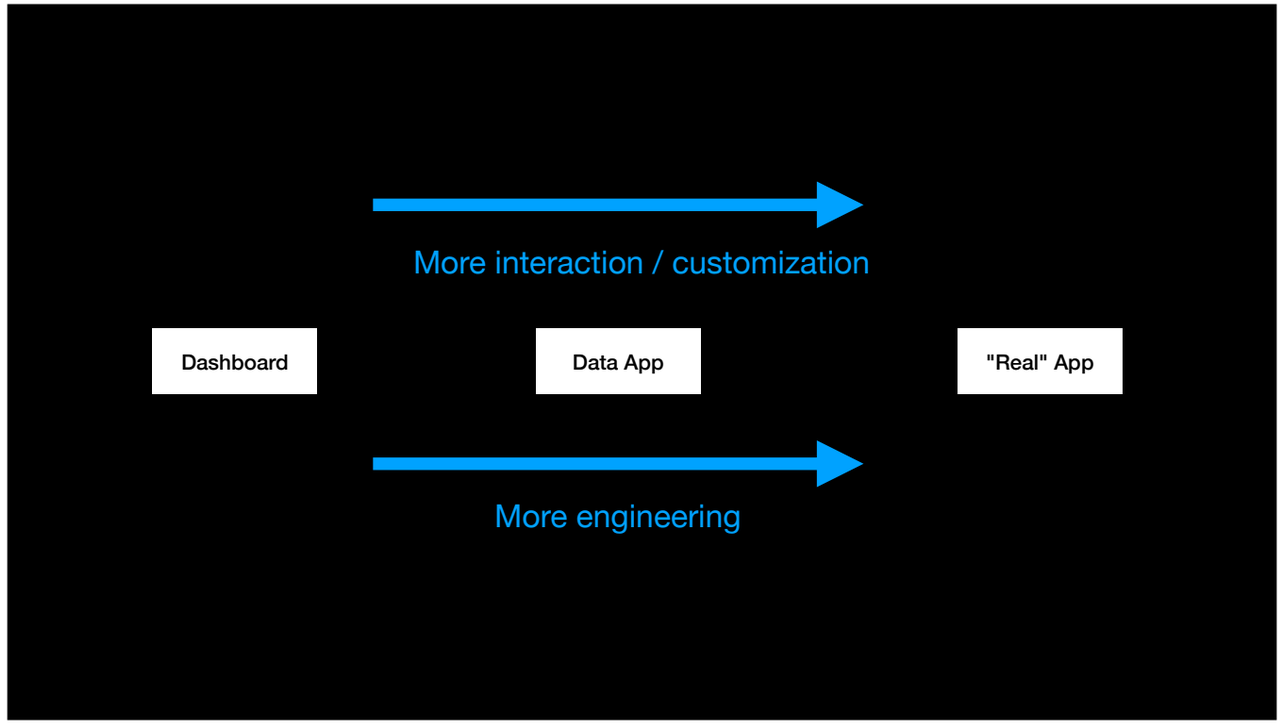
Exercise: What is a Data App?

- Type your answer to these 2 questions in the chat:
  - *What do you already know about Data Apps, Python and Streamlit?*
    - Streamlit is really popular: downloaded 16M times last month
  - *What do you want to build after completing this course?*
    - I use Streamlit to build data apps that make it easy to understand public / government datasets. This is the course I wish I had when I was starting out!

This is my "short answer" to the question. On the next slide I'll give a longer question.

Exercise: What is a Data App?

- My answer has two parts
  - "Data" apps vs."Real" apps vs. Dashboards
  - Data apps vs. Jupyter Notebooks



- A Dashboard has no interactivity whatsoever - think of the dashboard of car. And as a result, there are a lot of ways to create them with using little to no code.
- A "real" app, like Amazon, has a lot of interaction: creating an account, browsing items, purchase items and leave reviews. As a result, there is a lot of engineering involved.
- Data Apps are somewhere in the middle. Like a dashboard, they're designed to inform. But unlike a dashboard, you can interact with them. But unlike a "Real" app, that interaction is typically limited to exploring a dataset.
- Data apps require *some* engineering. In this course, we'll be writing small amounts of Python code. Streamlit will take care of all the front end code (like HTML, CSS and JavaScript). We won't be using a database / writing SQL.

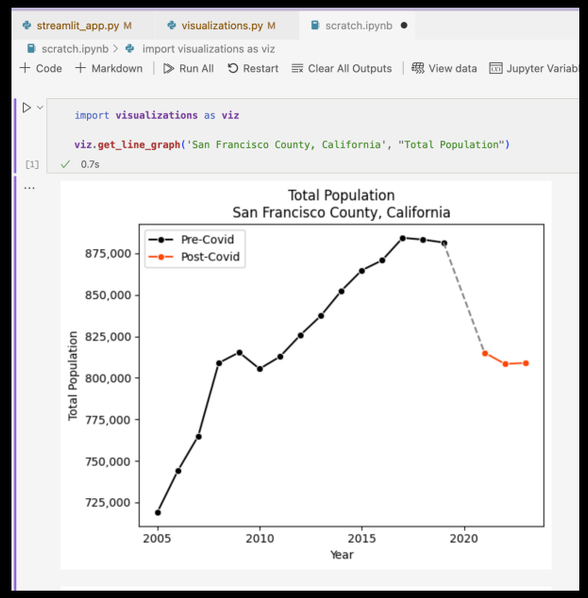
Data Apps vs. Jupyter Notebooks

- Jupyter Notebooks let me create a single story about a dataset. I can share the notebook with others for free via github.
- Streamlit lets me create an app to explore a dataset. I can share the app with others for free via Streamlit Cloud.
- I often start with a notebook, and end with a Streamlit app.

Ask: How many people here have used Jupyter notebooks before?
In my own work, I view Streamlit as a companion to Jupyter notebooks.

Example Jupyter Notebook

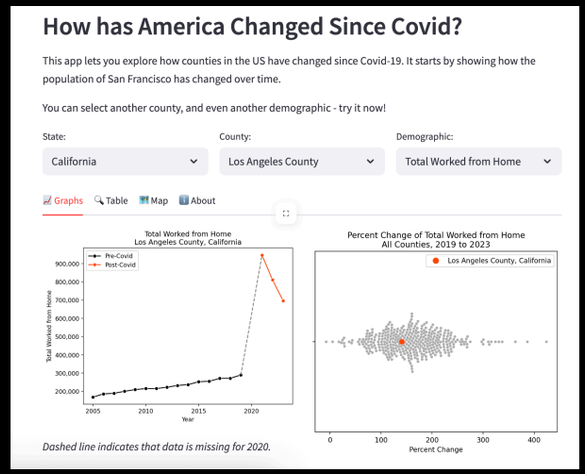
- This notebook tells a story about one city.



At one point I was interested in understanding how San Francisco's population changed during Covid. My first step was to create a Jupyter Notebook that had graphs like this.

Example Streamlit App

- This Streamlit App lets users see how the population of their county changed
- https://census-explorer.streamlit.app/



Note the interactivity:

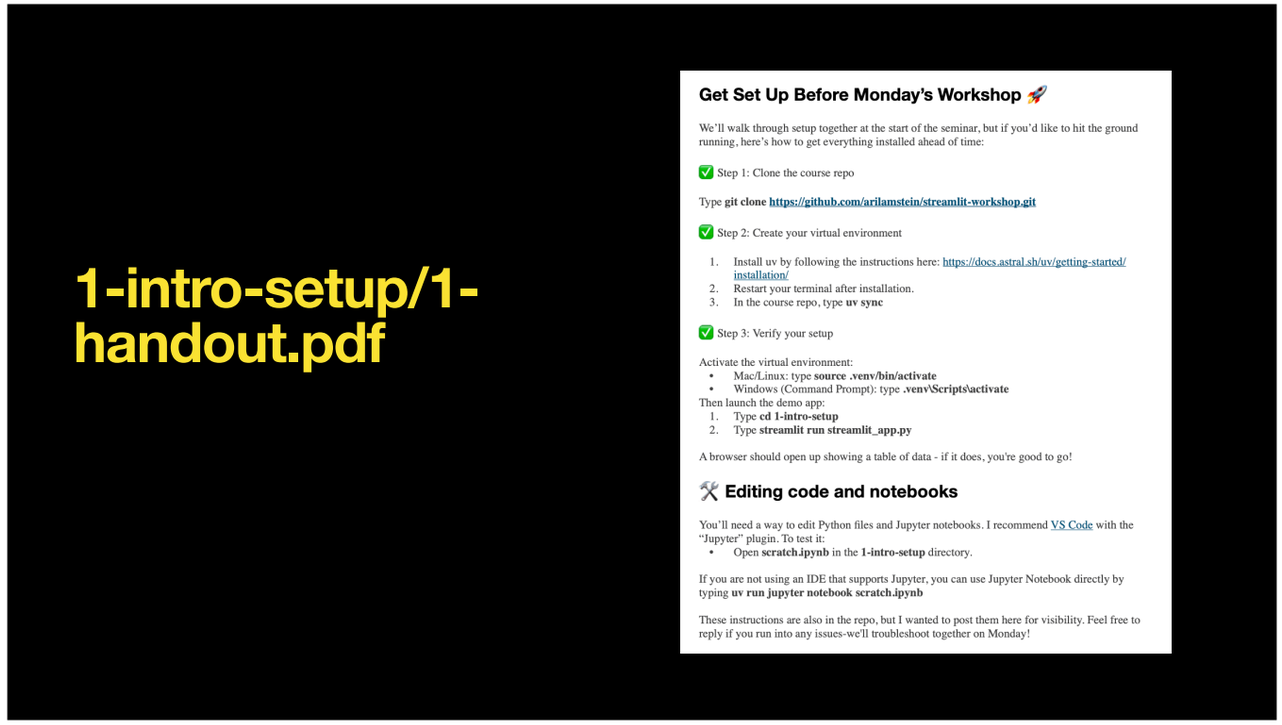
-select boxes -> graphs change.

-layout: tabs are clickable

Course Agenda
(Big Picture)

1. Session 1: Setup & Basics
2. Session 2: "The Loop": Inputs & Graphics
3. Session 3: Organization: UI & Code
4. Session 4: Deployment

I mentioned earlier that because there is some interaction and customization to Streamlit apps, they require some engineering. This engineering work normally has 3 components: setup (which we'll be covering in this section), development (where you build the app locally on your machine) and deployment (where you make the app publicly available).



The setup instructions are also in 1-intro-setup/1-handout.pdf.

Session 1: Setup

1. One-Time Setup
    1. Clone course repo

The first part of the setup is to get the course code onto your local machine. This is called cloning the course repository.

Clone course repo

- The course repo is at URL is at https://github.com/arilamstein/streamlit-workshop
- To clone it, go to the terminal and type

  `git clone https://github.com/arilamstein/streamlit-workshop.git`

```
➜  test git clone https://github.com/arilamstein/streamlit-workshop.
Cloning into 'streamlit-workshop'...
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 12 (delta 0), reused 12 (delta 0), pack-reused 0 (from
Receiving objects: 100% (12/12), 175.62 KiB | 1.66 MiB/s, done.
➜  test ls
streamlit-workshop
```

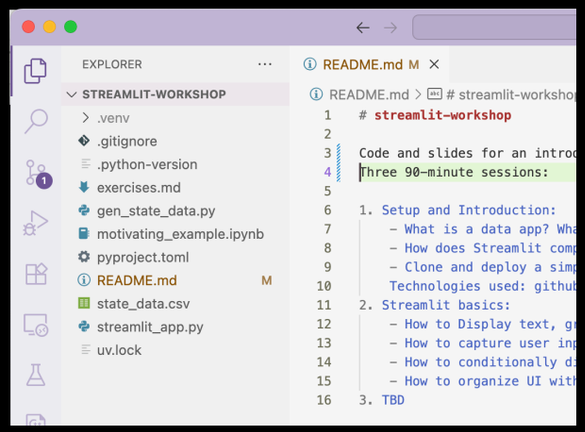
In order to get the course repo on your local machine, you will need to clone it.

Open the terminal to the directory where you want the repository and type "git clone https://github.com/arilamstein/streamlit-workshop.git".

If it worked correctly, you should see a new subdirectory called "streamlit-workshop"

Open Course Repo

- Open the course repo using your favorite IDE



Open up the "streamlit-workshop" directory in your favorite IDE. Here I'm using VS Code.

Session 1: Setup

1. One-Time Setup
    1. Clone course repo
    2. Create the virtual environment

Now that you have the code installed, it's time to create a virtual environment for the course. A virtual environment is basically a version of Python that is tied to a specific project.

uv

- "An extremely fast Python package and project manager, written in Rust"
- Uses 3 files:
  - pyproject.toml
  - uv.lock
  - .python-version

We'll be using a program called 'uv' to create the virtual environment. It's a new Python package manager that is taking the Python world by storm. It will install everything faster than older systems, and with less steps. Let's look at each of these files, one at a time. These files will also be used when we deploy our app to Streamlit Cloud.

pyproject.toml

- Specifies a (minium) version of Python for the project
- Specifies packages that need to be installed ("dependencies")
- Specifies minimum versions required

pyproject.toml

```toml
[project]
name = "streamlit-workshop"
version = "0.1.0"
description = "Add your descr
readme = "README.md"
requires-python = ">=3.13"
dependencies = [
    "censusdis>=1.4.2",
    "ipykernel>=6.30.1",
    "kaleido>=1.1.0",
    "pandas>=2.3.2",
    "plotly>=6.3.0",
    "streamlit>=1.49.1",
]
```

pyproject.toml defines your project's dependencies and configuration. It specifies the minimum Python version to use (in this case, >=3.13) and lists the packages your project needs under "dependencies", along with their minimum versions. You can open this file now—it's designed to be human-edited (but please don't edit it!)

uv.lock

- Says the exact version of each package to install
- Ensures reproducible environments
- Eliminates a common source of errors

```
[[package]]
name = "streamlit"
version = "1.49.1"
source = { registry = "https://pypi.org/simple" }
dependencies = [
    { name = "altair" },
    { name = "blinker" },
    { name = "cachetools" },
    { name = "click" },
    { name = "gitpython" },
    { name = "numpy" },
    { name = "packaging" },
    { name = "pandas" },
    { name = "pillow" },
    { name = "protobuf" },
    { name = "pyarrow" },
    { name = "pydeck" },
    { name = "requests" },
    { name = "tenacity" },
    { name = "toml" },
    { name = "tornado" },
    { name = "typing-extensions" },
    { name = "watchdog", marker = "sys_platform != 'darwin'" },
]
sdist = { url = "https://files.pythonhosted.org/packages/24/17/c8024
wheels = [
    { url = "https://files.pythonhosted.org/packages/85/9e/146cdef51
]
```

uv.lock records the exact versions of all packages used in your project, including sub-dependencies. It’s automatically generated by uv and ensures that your environment is fully reproducible. This helps eliminate a common source of errors—especially during workshops—by making sure everyone is using the same versions.

.python-version

- Says which version of Python to install
  - Major and minor version are specified
  - uv will always install the latest patch version
- This eliminates a common source of errors

.python-version

```
3.13
```

uv also lets you "pin" a version of Python to use. Again, this mimimizes the chance of something working on my computer but not on yours.

Install uv

- Google "install uv".
- The first hit should be this page.
- Copy the command into your terminal.

Getting started

Installing uv

Installation methods

Install uv with our standalone installers or your package manager

Standalone installer

uv provides a standalone installer to download and install uv:

macOS and Linux Windows

Use `curl` to download the script and execute it with `sh`:

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

Copy the url into the chat: https://docs.astral.sh/uv/getting-started/installation/

Installing uv

- What I see after installation

```
→  streamlit-workshop git:(main) ✗ curl -LsSf https://astral.sh/uv/install.sh | sh
downloading uv 0.8.17 aarch64-apple-darwin
no checksums to verify
installing to /Users/arilamstein/.local/bin
  uv
  uvx
everything's installed!
WARN: The following commands are shadowed by other commands in your PATH: uv uvx
```

Hopefully you don't get the same warnings I do :)

Restart your terminal

This is something that tripped up people in the last workshop I ran. After installing uv, you need to restart your terminal before using it.

uv sync

- In the terminal, go back to the project directory
- Type uv sync in the terminal
- Creates a virtual environment in the folder .venv that is "in sync" with the pyproject.toml, uv.lock and .python-version files
  - This directory will have the correct version of Python, and the correct version of each package the project needs

```
→  streamlit-workshop git:(main) ✗ uv sync
Using CPython 3.13.7
Creating virtual environment at: .venv
Resolved 145 packages in 4ms
Installed 139 packages in 385ms
 + adjusttext==0.8
 + affine==2.4.0
 + alabaster==1.0.0
 + altair==5.5.0
 + appnope==0.1.4
 + asttokens==3.0.0
 + attrs==25.3.0
 + babel==2.17.0
 + beautifulsoup4==4.13.5
 + black==25.1.0
 + bleach==6.2.0
 + blinker==1.9.0
 + cachetools==6.2.0
 + censusdis==1.4.2
 + certifi==2025.1.31
```

You can see the versions of the packages being installed in the output from uv sync.

Session 1: Setup

1. One-Time Setup
    1. Clone course repo
    2. Create the virtual environment
2. Per-Session Setup
    1. Activate the virtual environment
    2. Run the example Streamlit app

Cloning a repo only needs to be done once. And creating the virtual environment also only needs to be done once.

But each time you open the terminal, you'll need to activate the virtual environment. "Activating" the virtual environment just means "When I type 'python' in the terminal, it uses the version of python in the project's .venv directory."

Also, each time you want to run the app, you'll need to type a specific command.

Activate the Virtual Environment

- Activate the virtual environment:
  - **Mac/Linux**: source .venv/bin/activate
  - **Windows (Command Prompt)**: .venv\Scripts\activate

The command to activate the virtual environment depends on your operating system. Type the right command now

Test the Virtual Environment

- Check that the virtual environment is running:
  - Type **python check_env.py**

```
(streamlit-workshop) ➜  streamlit-workshop git:(main) ✗ python check_env.py
Python executable: /Users/arilamstein/uv-projects/streamlit-workshop/.venv/bin/python
Virtual env active: True
(streamlit-workshop) ➜  streamlit-workshop git:(main) ✗
```

Before we move on, please verify that your virtual environment is running. To do this, type "python check_env.py". This will run a test that should say "Virtual env active: True".

Session 1: Setup

1. One-Time Setup
    1. Clone course repo
    2. Create the virtual environment
2. Per-Session Setup
    1. Activate the virtual environment
    2. Run the example Streamlit app

Each time you open the terminal, you'll need to activate the virtual environment. And each time you want to run a streamlit app, you'll need to type a specific command.

Run the Demo App

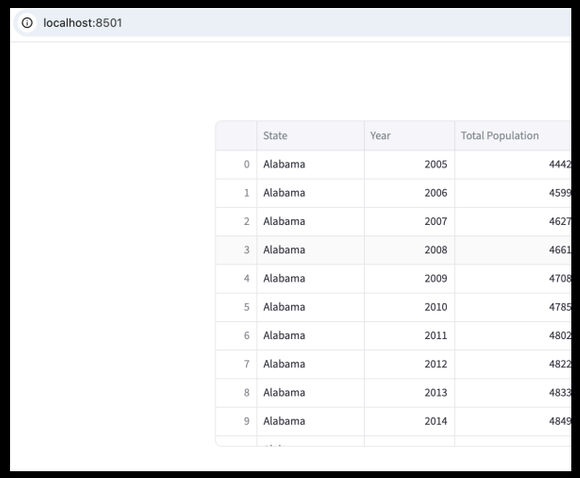
- Each session has its own directory
- Type `cd 1-intro-setup`
  - This moves to the directory for this session
- Type `streamlit run streamlit_app.py`
  - This runs the demo app for this section

```
(streamlit-workshop) ➜  streamlit-workshop git:(main) ✗ cd 1-intro-setup
(streamlit-workshop) ➜  1-intro-setup git:(main) ✗ streamlit run streamlit_app.py

  You can now view your Streamlit app in your browser.
```

Move to the "1-intro-setup" directory and type "streamlit run streamlit_app.py"

Running the App

- Type streamlit run streamlit_app.py
- It should open a browser like this
- Click the table headers to sort (e.g. State, Year, Total Population)
- **Quiz:**
  - What are the min and max years of the dataset?
  - What state had the lowest population?
  - What state had the highest income?



	State	Year	Total Population
0	Alabama	2005	4442
1	Alabama	2006	4599
2	Alabama	2007	4627
3	Alabama	2008	4661
4	Alabama	2009	4708
5	Alabama	2010	4785
6	Alabama	2011	4802
7	Alabama	2012	4822
8	Alabama	2013	4833
9	Alabama	2014	4849

Recap: Setup

1. One-Time Setup
    1. Clone course repo
    2. Create the virtual environment
2. Per-Session Setup
    1. Activate the virtual environment
    2. Run the example Streamlit app

Cloning the repo and creating the virtual environment were one-time actions.
Each time you create a new terminal, you'll need to activate the virtual environment.
And each time you want to run the app, you'll need to type "streamlit run <file>"

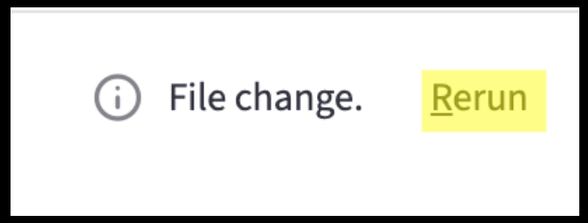
Course Agenda
(Big Picture)

1. Session 1: Setup & Basics

Now that you have a simple Streamlit app running, let's run through some basic things that will help you throughout the course.

Rerun

- Streamlit is designed for fast iteration
- Each time you modify and save the script the word Rerun will appear in the top right of the app
- Click it to rerun the app with your changes
- Do not stop and restart the app from the command line



Exercise: Rerun

- Open the file streamlit_app.py in the directory 1-intro-setup
- Type streamlit run streamlit_app.py to run the app
- Uncomment the st.title line
- Save the file
- Click "rerun" and verify that the title appears

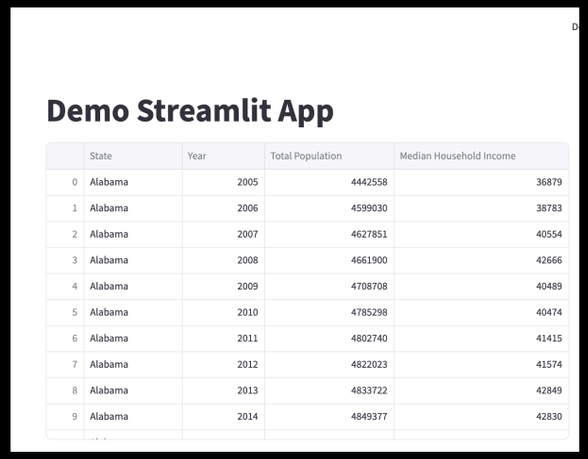
session1 > streamlit_app.py > ...

```python
import streamlit as st
import pandas as pd

#st.title("Demo Streamlit App")

df = pd.read_csv("state_data.csv")

st.dataframe(df)
```

This is an exercise to make sure you're comfortable changing the app, and clicking "rerun" to see your changes.

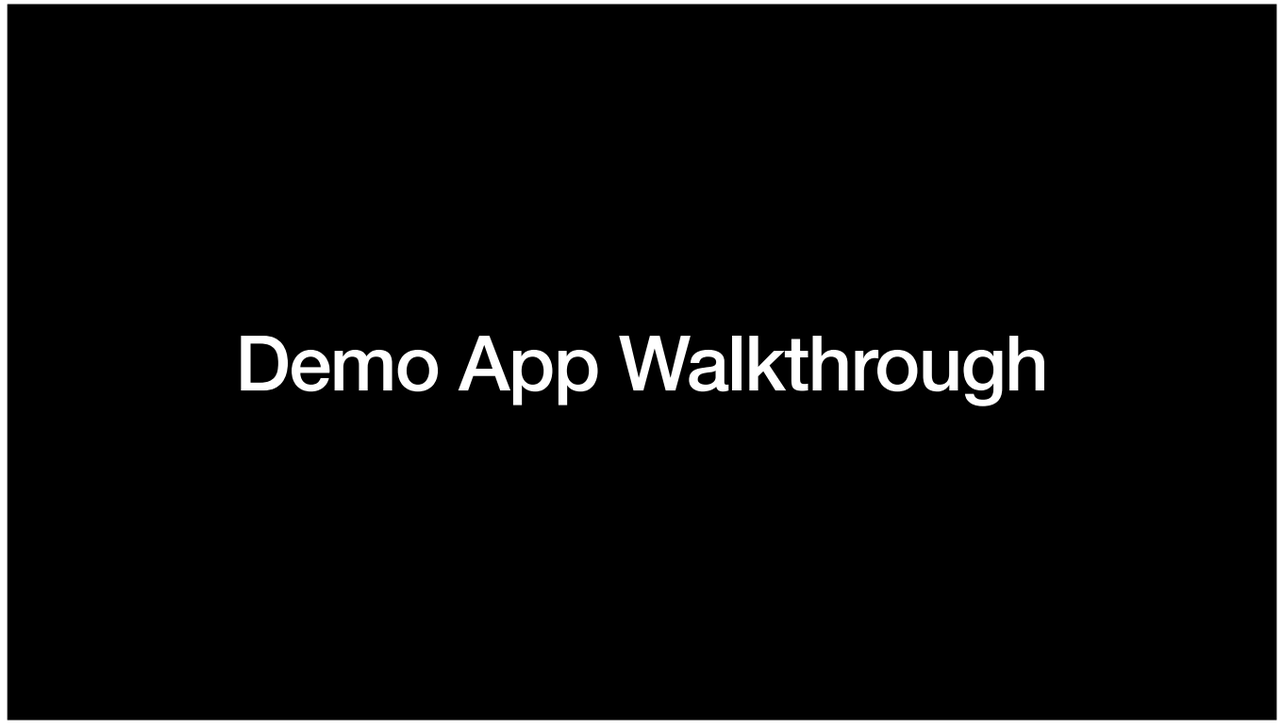
Exercise: Rerun

- Result



Demo Streamlit App

	State	Year	Total Population	Median Household Income
0	Alabama	2005	4442558	36879
1	Alabama	2006	4599030	38783
2	Alabama	2007	4627851	40554
3	Alabama	2008	4661900	42666
4	Alabama	2009	4708708	40489
5	Alabama	2010	4785298	40474
6	Alabama	2011	4802740	41415
7	Alabama	2012	4822023	41574
8	Alabama	2013	4833722	42849
9	Alabama	2014	4849377	42830



Before we start digging into real streamlit, let's cover what this basic app does. We'll be building on this app during the next session, so it pays to make sure we all know how the existing app works.

The data

- state_data.csv
- Basic demographic data of US States
- Each state, 2005-2023
- Comes from the American Community Survey (ACS)

session1 > state_data.csv > data

```
State,Year,Total Population,Median Household Income
Alabama,2005,4442558,36879
Alabama,2006,4599030,38783
Alabama,2007,4627851,40554
Alabama,2008,4661900,42666
Alabama,2009,4708708,40489
Alabama,2010,4785298,40474
Alabama,2011,4802740,41415
Alabama,2012,4822023,41574
Alabama,2013,4833722,42849
Alabama,2014,4849377,42830
Alabama,2015,4858979,44765
Alabama,2016,4863300,46257
```

Pandas

- The app uses the Pandas library to read in the csv file.
- Pandas is the most popular library in Python for working with tabular data
- Pandas is typically imported as pd
- pd.read_csv returns a dataframe

session1 > streamlit_app.py > ...

```python
import streamlit as st
import pandas as pd

#st.title("Demo Streamlit App")

df = pd.read_csv("state_data.csv")

st.dataframe(df)
```

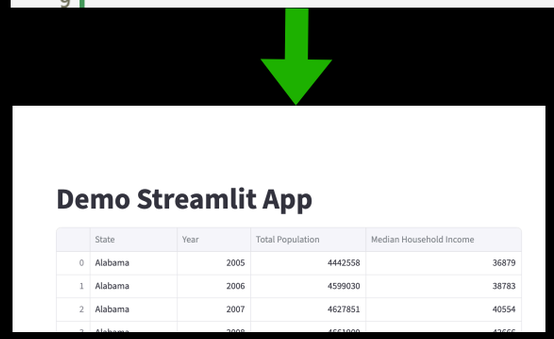
How many people have experience with Pandas?

Streamlit

- Lets the script run as a webapp
- Cannot run in a Jupyter notebook - must be a regular .py file.
- Imported as st
- Two output functions:
  - st.title
  - st.dataframe
- Processed top to bottom.

session1 > streamlit_app.py > ...

```python
import streamlit as st
import pandas as pd

st.title("Demo Streamlit App")

df = pd.read_csv("state_data.csv")

st.dataframe(df)
```



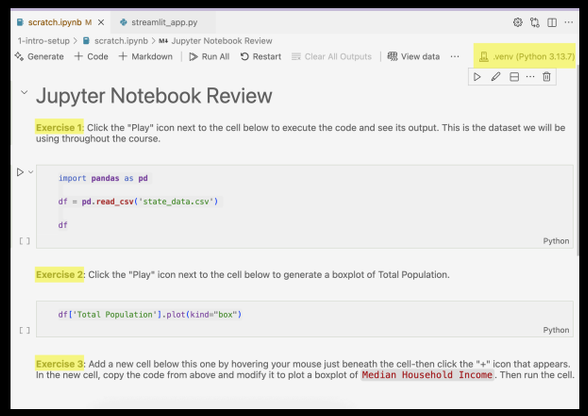
Our demo app also has the line "import streamlit as st".

Exercise: Jupyter Notebooks

- Open scratch.ipynb. Two ways:
  1. VS Code with Jupyter plugin
  2. Jupyter notebook via command line:

     uv run jupyter notebook scratch.ipynb

- Complete the 3 exercises as a review



Later on we'll be making a lot of use of Jupyter Notebooks. So I want to include this quick exercise so everyone is comfortable with them. Note that in VS Code you'll need to select a "kernel" in the upper right hand corner.

Section 1 Review

- What is a data app?
- 1-time setup:
  - Clone course repo
  - Create virtual environment
- Per-session setup:
  - Activate virtual environment
  - Run app
- Rerun app without restarting Streamlit
- Review technical basics: Pandas, Jupyter Notebooks, Streamlit